

Ride-Hailing

Base de Datos

Bases de Datos Avanzadas — Grupo 3A

Javier Picazo · Alejandro Bernaldo de Quirós
Pablo Cerdeira · Jaime Ordovás

MySQL 8.0 · InnoDB · Docker

Problema y caso de uso

1

Rider solicita

Introduce origen y destino
(geolocalización)

2

Se generan ofertas

La solicitud llega a varios
conductores disponibles

3

Primer conductor acepta

`SELECT...FOR UPDATE`
garantiza que solo uno se
queda el viaje

4

Viaje + Pago

El ciclo finaliza y se genera un
único pago

Estados del viaje:

solicitado

aceptado

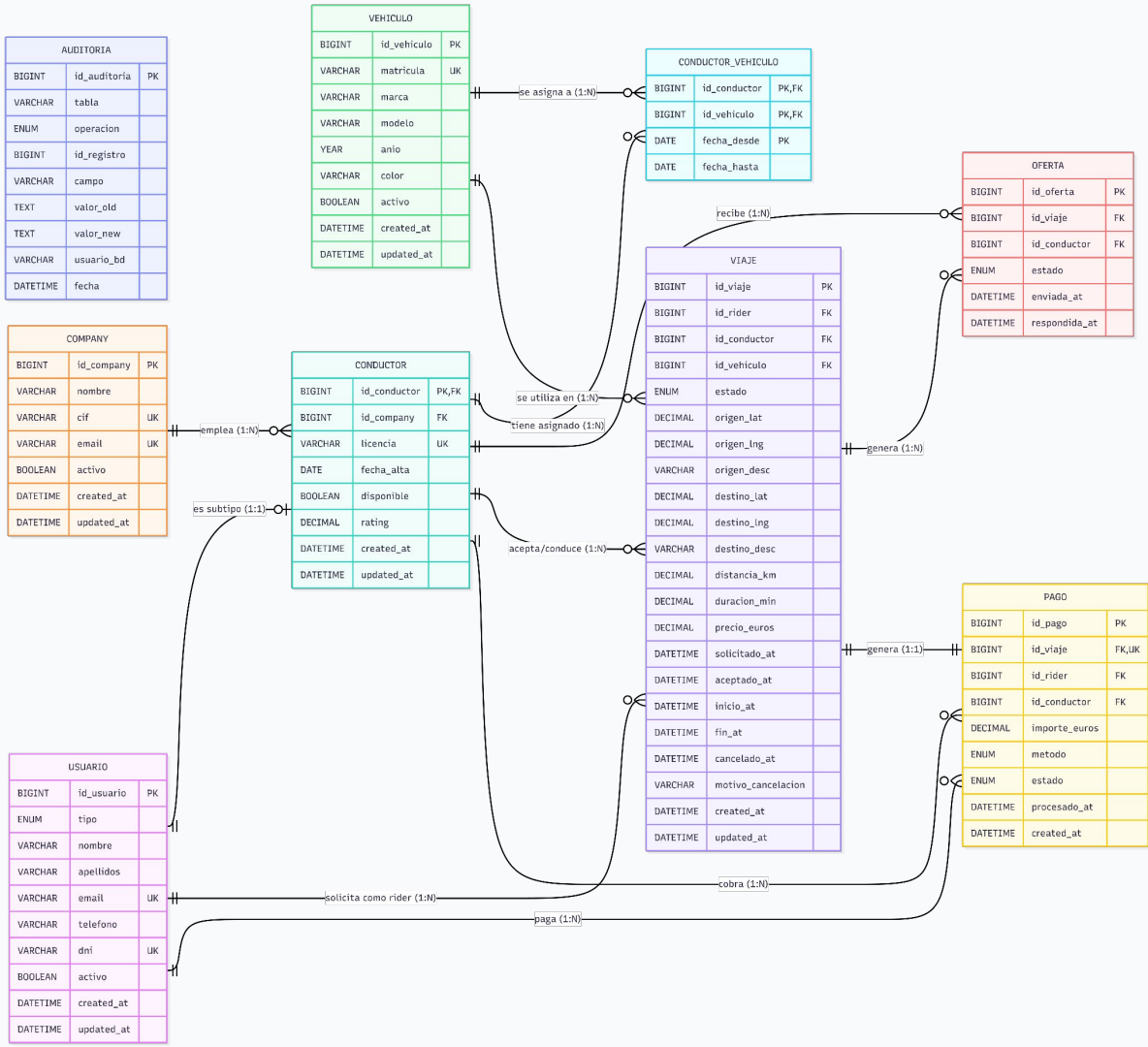
en_curso

finalizado

cancelado

Modelo Entidad-Relación

<https://mermaid.ai/app/projects/9d72dda2-e5b4-400e-8249-861b2ac4c2e0/diagrams/fe3143f1-8084-4c9e-8b7e-b5f4bc5cf011/version/v0.1/edit>



Decisiones de diseño principales

01

Herencia usuario → conductor

Tabla única `usuario` con campo tipo ENUM. `conductor` extiende con PK=FK (1:1). Unicidad global de email y DNI, sin duplicar columnas.

02

N:N temporal conductor ↔ vehículo

PK compuesta (id_conductor, id_vehiculo, fecha_desde). fecha_hasta NULL = asignación vigente. Historial completo de qué vehículo usó cada conductor.

03

Ciclo de vida con timestamps

5 estados ENUM + 5 timestamps independientes. Permite calcular esperas y duración reales.

04

Pago 1:1 con viaje

UNIQUE KEY sobre id_viaje en la tabla pago. Garantiza exactamente un pago por viaje finalizado. Simplifica la contabilidad.

05

Borrado lógico

Campo activo BOOLEAN. ON DELETE RESTRICT en todas las FKs. Las ofertas se marcan 'expirada', nunca se borran.

06

Auditoría con triggers

Triggers AFTER INSERT/UPDATE. Operador NULL-safe <=> para comparar valores nulos. Registro automático en tabla auditoria.

Índices y Vistas

Índices

<code>idx_viaje_estado</code>	viaje	Filtro más frecuente en dashboards
<code>idx_viaje_estado_fecha</code>	viaje	Compuesto: estado + solicitado_at
<code>idx_oferta_viaje_estado</code>	oferta	Búsqueda en sp_aceptar_oferta
<code>idx_conductor_disponible</code>	conductor	Conductores disponibles para ofertas
<code>idx_pago_conductor</code>	pago	Ingresos por conductor (compuesto)
<code>idx_viaje_rider / conductor</code>	viaje	FK para JOINS
<code>idx_audit_tabla_id / fecha</code>	auditoria	Consultas de auditoría

Vistas

`v_viajes_detalle`

Desnormaliza viaje + rider + conductor + company + vehículo. Simplifica el dashboard.

`v_conductor_publico`

Solo datos no sensibles del conductor. Sin DNI ni email. Patrón de seguridad.

`v_metricas_conductor`

KPIs por conductor: viajes, ingresos, km medio, €/km, €/min, tasa aceptación.

`v_metricas_company`

KPIs por company: nº conductores, ingresos, tasa aceptación.

Concurrencia: sp_aceptar_oferta

¿Qué pasa si dos conductores aceptan el mismo viaje al mismo tiempo?

01

START TRANSACTION

Se abre una transacción. Todo lo que siga es atómico.

02

SELECT ... FOR UPDATE

Bloqueo exclusivo sobre la fila de la oferta. El segundo conductor queda en espera.

03

¿Estado = pendiente?

Si ya fue aceptada o expirada → ROLLBACK y mensaje de error.

04

SELECT ... FOR UPDATE

Bloqueo exclusivo sobre el viaje. Verifica que estado = 'solicitado'.

05

Actualizar oferta y viaje

Oferta → 'aceptada'. Viaje → 'aceptado'.
Resto → 'rechazadas'.

06

COMMIT

Se liberan los locks. El segundo conductor ve el cambio y recibe ERROR.

Seguridad: roles y permisos

rol_app

api_app@%

- SELECT, INSERT, UPDATE en **ridehailing.***
- DELETE en **oferta** (expirar)
- EXECUTE **stored procedures**

API backend. Sin DDL, sin borrado físico. Borrado siempre lógico.

rol_analytics

bi_reports@%

- **SELECT SOLO** sobre vistas
- `v_viajes_detalle`
- `v_metricas_conductor`
- `v_metricas_company`

Reporting. Nunca accede a tablas directas. Oculta DNI y email.

rol_backup

backup_user@localhost

- SELECT + RELOAD
- LOCK TABLES
- REPLICATION CLIENT
- SHOW VIEW, EVENT

`mysqldump --single-transaction`. Solo desde localhost.

rol_dba

dba_admin@localhost

- **ALL PRIVILEGES** en **ridehailing.***
- PROCESS, RELOAD
- REPLICATION CLIENT
- Solo localhost

Administración. Nunca se expone directamente como root.

Principio de mínimo privilegio: cada usuario solo tiene los permisos estrictamente necesarios para su función.

Backup y recuperación

1h

RPO

Máxima pérdida de datos

4h

RTO

Tiempo máximo para restaurar

7d

Retención

Backups + binlogs

Estrategia de backup:

03:00 AM



mysqldump diario

```
--single-transaction  
--routines --triggers  
--set-gtid-purged=OFF
```

Continuo



Binlog ROW

Captura todos los cambios entre backups

Si hay fallo



Restore + PITR

1. Restaurar backup
2. mysqlbinlog hasta el minuto anterior

backup.sql incluye: verificación de binlog, integridad post-restore (UNION de conteos + FKs) y comandos comentados de PITR.

Dashboards de monitorización

Dashboard BD (dashboard.sql)

DB-1 Tamaño por tabla (MB) `information_schema.tables`

DB-2 Conexiones activas vs máximo `performance_schema.global_status`

DB-3 Buffer pool hit ratio (debe ser >99%) `Innodb_buffer_pool_reads`

DB-4 Queries por tipo (SELECT/INSERT/...) `Com_select, Com_insert...`

DB-5 Deadlocks y row lock waits `Innodb_deadlocks`

DB-6 Transacciones activas ahora `information_schema.INNODB_TRX`

DB-7 Top 10 queries más lentas `performance_schema.events_statements...`

DB-8 Índices no usados `sys.schema_unused_indexes`

Dashboard Negocio

BIZ-1 KPIs globales: total viajes, cancelados, ticket medio, ingresos

BIZ-2 Viajes por hora (últimas 24h)

BIZ-3 Tasa aceptación de ofertas por día (7 días)

BIZ-4 Top 5 conductores por ingresos (30 días)

BIZ-5 Métricas por company (via vista `v_metricas_company`)

BIZ-6 Tiempo medio de espera rider (solicitud → inicio)

BIZ-7 Distribución de viajes por franja horaria

BIZ-8 Riders más activos (top 10)

Despliegue con Docker Compose

docker-compose.yml

mysql:8.0.40

container: ridehailing-db

ports: 3306:3306

volumes (init):

- schema.sql
- permissions.sql
- data.sql

config:

- mysql/conf.d/custom.cnf

healthcheck: enabled

adminer:latest

ridehailing-adminer

ports: 8080:8080

depends_on: mysql

DEFAULT_SERVER: mysql

mysql_data volume

Persistencia de datos

custom.cnf

binlog ROW · slow_log · InnoDB

Comandos clave

Arrancar

```
docker compose up -d
```

Estado

```
docker compose ps
```

Conectar

```
docker exec -it ridehailing-db  
mysql -uroot -prootpass
```

Dashboard

```
docker exec -i ridehailing-db  
mysql ... < dashboard.sql
```

Reset

```
docker compose down -v  
docker compose up -d
```

Equipo y aportaciones



Javier Picazo

dashboard.sql — métricas de BD y de negocio

queries.sql — stored procedures y concurrencia

Presentación y revisión



Alejandro Bernaldo

permissions.sql — roles, usuarios y permisos

compose.yml — configuración Docker y Adminer

README.md — instrucciones de despliegue

Presentación y revisión



Pablo Cerdeira

backup.sql — estrategia RPO/RTO y PITR

DESIGN.md — documentación técnica y MER

Presentación y revisión



Jaime Ordovás

schema.sql — diseño de tablas, índices, vistas y triggers

data.sql — carga masiva y SP de generación de datos

Presentación y revisión

Gracias

¿Alguna Pregunta?